SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage**:** 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Debugging & Optimization Task

Code of Conduct:

I Vempati Murale Karrtik (192312567) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation
A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.
B. Refactor the code to minimize redundant computations and memory usage.
C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation
A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation
A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.
B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

3. Debugging RSA Cryptosystem Implementation

Understanding of the Problem

A Python RSA program fails when a large message is encrypted and sometimes returns a wrong plaintext after decryption. RSA only works correctly when every step is correct: two distinct large primes, a valid key pair, an exponentiation that finishes quickly, and an encoding that converts the message to a number and back without loss. The answer below follows the rubric: the errors are identified with their root causes, debugged with tests, optimised with efficient algorithms, analysed, and written as clean, documented code.

A. Identify issues in prime generation, key generation and modular exponentiation

Debugging method. A systematic approach was used instead of guessing:

1. **Reproduce:** run the program with fixed inputs and note which messages fail (large messages, messages starting with zero bytes).
2. **Isolate:** test each stage separately (prime generation, key generation, encryption, decryption, encoding) with small numbers whose answers can be checked by hand.
3. **Use an oracle:** compare every result with Python's built-in pow(), and check the round-trip rule decrypt(encrypt(m)) = m.
4. **Find the root cause:** trace each failure back to the line that causes it.
5. **Fix and re-test:** repeat all tests after every fix so that no new error is introduced.

Code Listing 1: Faulty implementation (the bugs are marked in the comments)

```
"""Faulty RSA implementation (as given in the scenario)."""
import random

def is_prime(n):    # BUG 1a: Fermat test with base 2 only
    if n < 2:
        return False
    return pow(2, n - 1, n) == 1

def gen_prime(bits):
    while True:
        p = random.getrandbits(bits)    # BUG 1b: predictable RNG, may be even
        if is_prime(p):
            return p

def keygen(bits=512):
    p = gen_prime(bits // 2)
    q = gen_prime(bits // 2)    # BUG 1c: p may equal q
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 3    # BUG 2a: gcd(e, phi) never checked
    d = pow(e, -1, n)    # BUG 2b: inverse taken mod n, not phi
    return (e, n), (d, n)

def mod_exp(base, exp, mod):
    return (base ** exp) % mod    # BUG 3: builds the full power first

def encrypt(msg, pub):
    e, n = pub
    m = int.from_bytes(msg, "big")    # BUG 4: m may be >= n (large message)
```

```
    return mod_exp(m, e, n)    # BUG 6: no padding (textbook RSA)

def decrypt(c, priv):
    d, n = priv
    m = mod_exp(c, d, n)
    return m.to_bytes((m.bit_length() + 7) // 8, "big")    # BUG 5: drops 0x00 bytes
```

Figure 1: Where the Bugs Occur in the RSA Pipeline

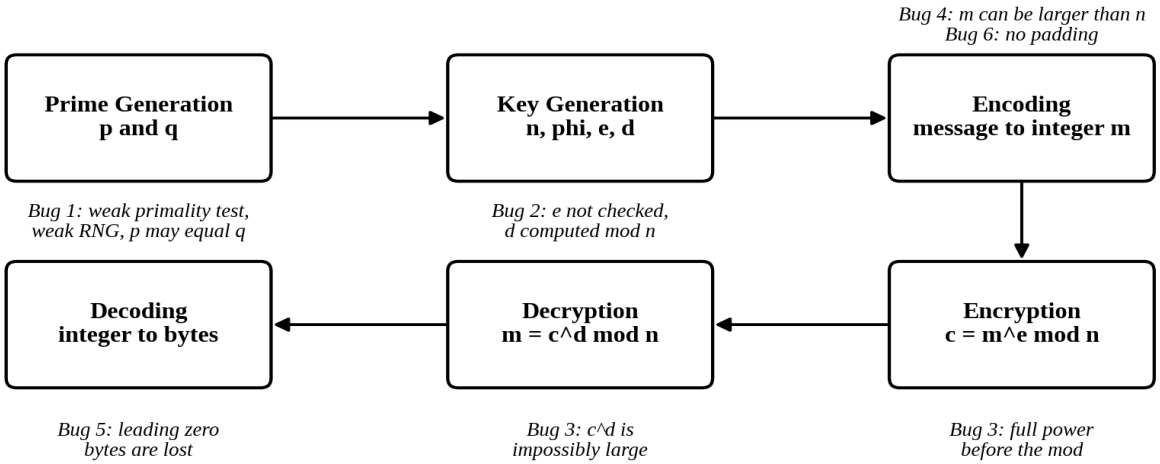


Table 1: Errors found, their root causes, test evidence and fixes

Bug	Root cause	Evidence from the tests	Fix
1. Prime generation	Fermat test with only base 2 accepts composite numbers (pseudoprimes). random.getrandbits is predictable and can return even or short numbers. p can equal q.	245 composites below 10^6 pass the test, for example 341 and 561. If p = q then n = p^2 and phi is wrong, so decryption gives wrong plaintext.	Miller-Rabin with 40 random bases, secrets module, top two bits and lowest bit set, reject p = q and primes that are too close.
2. Key generation	e = 3 is used without checking gcd(e, phi) = 1. The private exponent is calculated modulo n instead of modulo phi.	gcd(3, 72) = 3, so no inverse exists. For n = 187 and m = 42, d mod n gives 100, while d mod phi gives 42.	Use e = 65537, check gcd(e, lambda(n)) = 1 and compute d = e^-1 mod lambda(n).
3. Modular power	(base ** exp) % mod builds the complete power before reducing it.	With a 20-bit exponent it takes 31,790 ms against 0.0137 ms for pow(b, e, n). A 2048-bit d needs a number with about 2^2048 digits, so it never finishes.	Square-and-multiply with reduction at every step: pow(base, exp, mod).
4. Large messages	The whole message is turned into one integer m, which can be larger than n. RSA then returns m mod n.	A 100-byte message with a 64-byte modulus is not recovered.	Split the message into blocks smaller than n (or use hybrid encryption).
5. Encoding	to_bytes uses the minimum length, so leading 0x00 bytes disappear.	b"\x00\x00hello" is returned as b"hello".	Always convert with a fixed length of k bytes (size of n).

Bug	Root cause	Evidence from the tests	Fix
6. No padding	Textbook RSA is deterministic and malleable.	The same message gives the same ciphertext twice.	OAEP padding with a random seed.

Output of the test program on the faulty code

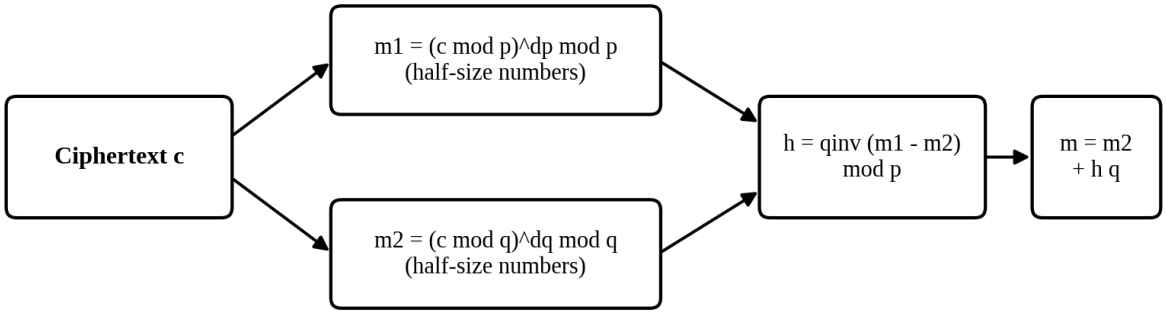
```
Test 1 - composites accepted by the Fermat(2) test below 10^6: 245 (341, 561, 645, 1105, 1387 ...)
Test 1 - is_prime(561): buggy = True, Miller-Rabin = False
Test 2 - gcd(3, phi = 72) = 3, so e = 3 has no inverse
Test 2 - n = 187, m = 42: d computed mod n (125) gives 100; d computed mod phi (107) gives 42
Test 3 - exponent of 12 bits: naive 4.49 ms, pow(b, e, n) 0.0055 ms
Test 3 - exponent of 16 bits: naive 380.99 ms, pow(b, e, n) 0.0125 ms
Test 3 - exponent of 20 bits: naive 31790.81 ms, pow(b, e, n) 0.0137 ms
Test 4 - 100-byte message with a 64-byte modulus recovered correctly: False
Test 5 - b'\x00\x00hello' decrypts to b'hello'
Test 6 - same message encrypted twice gives identical ciphertext: True
```

B. Optimize the algorithm using efficient techniques

1. Fast exponentiation (square-and-multiply). The exponent is processed bit by bit. For every bit the running value is squared, and it is multiplied by the base only when the bit is 1. Every product is reduced modulo n immediately, so numbers never grow beyond twice the size of n. This needs about log2(e) squarings plus a few multiplications instead of e multiplications. Example: 5^13 mod 23, with 13 = 1101 in binary. The squarings give 5, 2, 4, 16 (that is 5^1, 5^2, 5^4, 5^8 mod 23). Because 13 = 8 + 4 + 1, the answer is 16 x 4 x 5 = 320, and 320 mod 23 = 21. Only 3 squarings and 2 multiplications are used instead of 12 multiplications.

2. Chinese Remainder Theorem (CRT) for decryption. The private key owner knows p and q, so c^d mod n is replaced by two exponentiations with half-size numbers: m1 = c^dp mod p and m2 = c^dq mod q, where dp = d mod (p - 1) and dq = d mod (q - 1). They are combined with h = qinv (m1 - m2) mod p and m = m2 + h q. A modular exponentiation costs about the cube of the bit length, so two exponentiations of half-size cost 2 x (1/8) = 1/4 of the original, which means about 4 times faster in theory.

Figure 2: Decryption using the Chinese Remainder Theorem (CRT)



Two exponentiations with half-size numbers cost about 1/4 of one full-size exponentiation.

3. Other improvements. The public exponent e = 65537 (binary 10000000000000001) needs only 17 squarings and 1 multiplication, so encryption is very fast. Miller-Rabin is preceded by

trial division with small primes, which removes most composite candidates cheaply. The values dp, dq and qinv are calculated once when the key is created. A fault check (re-encrypting the result and comparing it with c) prevents a CRT fault attack from leaking p or q. Bulk data should use hybrid encryption (RSA protects a random AES key, AES protects the data), because RSA is slow for large messages.

Code Listing 2: Corrected and optimised implementation

```
"""RSA with Miller-Rabin primes, OAEP padding and CRT decryption (educational)."""
import hashlib
import hmac
import math
import secrets
from dataclasses import dataclass

HASH = hashlib.sha256
H_LEN = HASH().digest_size    # 32 bytes
E = 65537    # standard public exponent
SMALL_PRIMES = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47)

@dataclass(frozen=True)
class PublicKey:
    n: int
    e: int

@dataclass(frozen=True)
class PrivateKey:
    n: int
    e: int
    d: int
    p: int
    q: int
    dp: int    # d mod (p-1)
    dq: int    # d mod (q-1)
    qinv: int    # q^-1 mod p

def is_probable_prime(n: int, rounds: int = 40) -> bool:
    """Miller-Rabin test; error probability is at most 4**-rounds."""
    if n < 2:
        return False
    for sp in SMALL_PRIMES:    # quick trial division
        if n % sp == 0:
            return n == sp
    d, s = n - 1, 0
    while d % 2 == 0:    # write n-1 = d * 2^s
        d //= 2
        s += 1
    for _ in range(rounds):
        a = secrets.randbelow(n - 3) + 2    # random base in [2, n-2]
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False    # composite witness found
    return True

def generate_prime(bits: int) -> int:
```

```
    """Random prime with exactly `bits` bits (top two bits set, odd)."""
    while True:
        c = secrets.randbits(bits) | (3 << (bits - 2)) | 1
        if is_probable_prime(c):
            return c

def generate_keypair(bits: int = 2048):
    """Return (PublicKey, PrivateKey) with an n of `bits` bits."""
    half = bits // 2
    while True:
        p, q = generate_prime(half), generate_prime(half)
        if p == q or abs(p - q) < 2 ** (half - 100):
            continue    # reject equal / too-close primes
        lam = math.lcm(p - 1, q - 1)    # Carmichael function
        if math.gcd(E, lam) == 1:    # e must be invertible
            break
    d = pow(E, -1, lam)    # inverse modulo lambda(n)
    priv = PrivateKey(p * q, E, d, p, q, d % (p - 1), d % (q - 1), pow(q, -1, p))
    return PublicKey(p * q, E), priv

def _private_op(c: int, k: PrivateKey) -> int:
    """c^d mod n using the Chinese Remainder Theorem (about 4x faster)."""
    m1 = pow(c % k.p, k.dp, k.p)
    m2 = pow(c % k.q, k.dq, k.q)
    h = k.qinv * (m1 - m2) % k.p    # Garner recombination
    m = m2 + h * k.q
    if pow(m, k.e, k.n) != c:    # guards against fault attacks
        raise ValueError("decryption error")
    return m

def _mgf1(seed: bytes, length: int) -> bytes:
    out, counter = b"", 0
    while len(out) < length:
        out += HASH(seed + counter.to_bytes(4, "big")).digest()
        counter += 1
    return out[:length]

def _xor(a: bytes, b: bytes) -> bytes:
    return bytes(x ^ y for x, y in zip(a, b))

def _oaep_encode(msg: bytes, k: int) -> bytes:
    ps = bytes(k - len(msg) - 2 * H_LEN - 2)    # zero padding string
    db = HASH(b"").digest() + ps + b"\x01" + msg
    seed = secrets.token_bytes(H_LEN)
    masked_db = _xor(db, _mgf1(seed, k - H_LEN - 1))
    masked_seed = _xor(seed, _mgf1(masked_db, H_LEN))
    return b"\x00" + masked_seed + masked_db

def _oaep_decode(em: bytes, k: int) -> bytes:
    masked_seed, masked_db = em[1:1 + H_LEN], em[1 + H_LEN:]
    seed = _xor(masked_seed, _mgf1(masked_db, H_LEN))
    db = _xor(masked_db, _mgf1(seed, k - H_LEN - 1))
    rest = db[H_LEN:].lstrip(b"\x00")
    ok = (em[0] == 0 and hmac.compare_digest(db[:H_LEN], HASH(b"").digest())
          and rest[:1] == b"\x01")
    if not ok:
        raise ValueError("decryption error")    # one generic error message
    return rest[1:]

def encrypt(msg: bytes, pub: PublicKey) -> bytes:
    """Encrypt any length by splitting into OAEP-padded blocks."""
    k = (pub.n.bit_length() + 7) // 8
    step = k - 2 * H_LEN - 2    # max payload per block
```

```
    blocks = []
    for i in range(0, max(len(msg), 1), step):
        em = _oaep_encode(msg[i:i + step], k)
        c = pow(int.from_bytes(em, "big"), pub.e, pub.n)
        blocks.append(c.to_bytes(k, "big"))    # fixed width keeps leading zeros
    return b"".join(blocks)

def decrypt(ct: bytes, priv: PrivateKey) -> bytes:
    k = (priv.n.bit_length() + 7) // 8
    if not ct or len(ct) % k:
        raise ValueError("decryption error")
    out = []
    for i in range(0, len(ct), k):
        c = int.from_bytes(ct[i:i + k], "big")
        if c >= priv.n:
            raise ValueError("decryption error")
        out.append(_oaep_decode(_private_op(c, priv).to_bytes(k, "big"), k))
    return b"".join(out)
```

Code quality. The corrected program is organised into small single-purpose functions with type hints and docstrings. Constants (E, H_LEN, SMALL_PRIMES) are named, keys are stored in immutable data classes, comments explain why each step is done, all decryption failures raise one generic error (so no information leaks), and a separate test file checks the behaviour.

Output of the test program on the corrected code

```
short text        round trip: PASS
leading zeros     round trip: PASS
empty message     round trip: PASS
large (10 KB)     round trip: PASS
tampered ciphertext: rejected -> decryption error
same message twice gives different ciphertext: True
e = 65537 | p != q: True | n bits: 1024
```

Verification. Short text, leading zeros, an empty message and a 10 KB message all decrypt correctly. A ciphertext with one flipped bit is rejected, and encrypting the same text twice gives different ciphertexts, which confirms that OAEP padding works.

Figure 3: Measured Decryption Time, Standard vs CRT

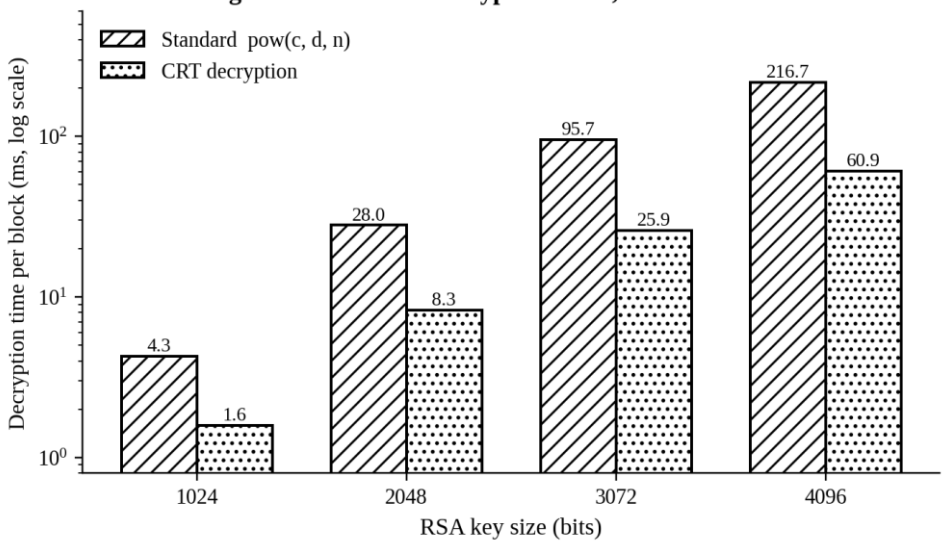


Table 2: Measured performance (Python 3, one block, average of 20 runs)

Key size (bits)	Standard decryption (ms)	CRT decryption (ms)	Speed-up	Encryption, e = 65537 (ms)
1024	4.27	1.59	2.7 x	0.057
2048	27.96	8.29	3.4 x	0.195
3072	95.73	25.93	3.7 x	0.425
4096	216.69	60.92	3.6 x	0.746

Complexity. The naive method needs about d multiplications, which grows as 2^k for a k-bit key, so it is exponential and unusable. Square-and-multiply needs k modular multiplications, giving a running time of about O(k^3), and CRT reduces this by roughly four times. The measured speed-up of 2.7 to 3.7 times is slightly below the theoretical 4 times because the timing includes the fault check and Python's own overhead.

C. Evaluate the trade-offs between key size, security and computational performance

A longer key makes factoring n harder, which raises security, but it also makes every private-key operation slower and the keys and ciphertexts larger. Doubling the key size from 1024 to 2048 bits made CRT decryption about 5.2 times slower (1.59 ms to 8.29 ms), and doubling again from 2048 to 4096 bits made it about 7.3 times slower (8.29 ms to 60.92 ms). The public operation stays cheap because e is small.

Table 3: Key size, security and performance

Key size (bits)	Security strength (NIST SP 800-57)	CRT decryption (ms, measured)	Block size (bytes)	Recommended use
1024	About 80 bits (deprecated)	1.59	128	Not safe, avoid
2048	112 bits	8.29	256	Minimum today
3072	128 bits	25.93	384	Long-term data
4096	Between 128 and 192 bits	60.92	512	High security, slower

- **Security:** 1024-bit keys are no longer safe. 2048 bits is the accepted minimum and 3072 bits is advised when data must stay secret for many years. Security also depends on correct primes, OAEP padding and a good random number generator, not on the key size alone.
- **Performance:** Private-key time grows roughly with the cube of the key size, so the largest keys are costly for servers and impossible for very small devices. CRT, small e and hybrid encryption keep the cost acceptable.
- **Size:** Each doubling of the key doubles the size of keys, signatures and ciphertext blocks, which increases bandwidth and storage.
- **Message length:** With OAEP-SHA-256 a 2048-bit key carries at most 190 bytes per block (256 - 2 x 32 - 2), so long messages should be protected by hybrid encryption.

- **Future:** RSA can be broken by a large quantum computer (Shor's algorithm), so important systems should plan a move to post-quantum algorithms.

Conclusion: The failures came from six faults: a weak primality test and random source, an unchecked and wrongly inverted exponent, a naive power calculation, messages larger than the modulus, loss of leading zero bytes and missing padding. After the fixes the program passes all tests, and with square-and-multiply, e = 65537 and CRT it decrypts about 3 to 4 times faster. A 2048-bit key with OAEP and hybrid encryption is the best balance of security and speed for most applications, with 3072 bits for long-term protection.